COMMON GOOD / FINANCIAL DAPP

Transparent giving. A visible fund trail.

Charity Donation Platform - 6113 Individual Assignment

Student details: to be completed by the student before submission.

Evidence date: 7 October 2026. This report describes a working local prototype and reproducible tests. The student must add the deployed Sepolia address, Render URL, public transaction hashes and wallet screenshots before final submission.

1. Introduction

Common Good is a financial decentralized application for small community donations. Users send Ethereum test ETH to a selected campaign through MetaMask. A Solidity contract records each contribution and restricts payouts to the campaign beneficiary. A Flask service makes contract state and receipts accessible through a responsive web interface. The project demonstrates programmable custody, wallet authorization and transparent financial records on a test network.

2. Problem statement

A donor viewing a simple fundraising page often cannot independently reconcile the displayed balance with contributions or see when funds were paid to a beneficiary. A conventional database may allow its operator to alter totals without a public audit trail. This prototype addresses the visibility of the financial flow: campaign-level contributions, retained funds and payouts can be checked against an Ethereum contract and transaction receipts.

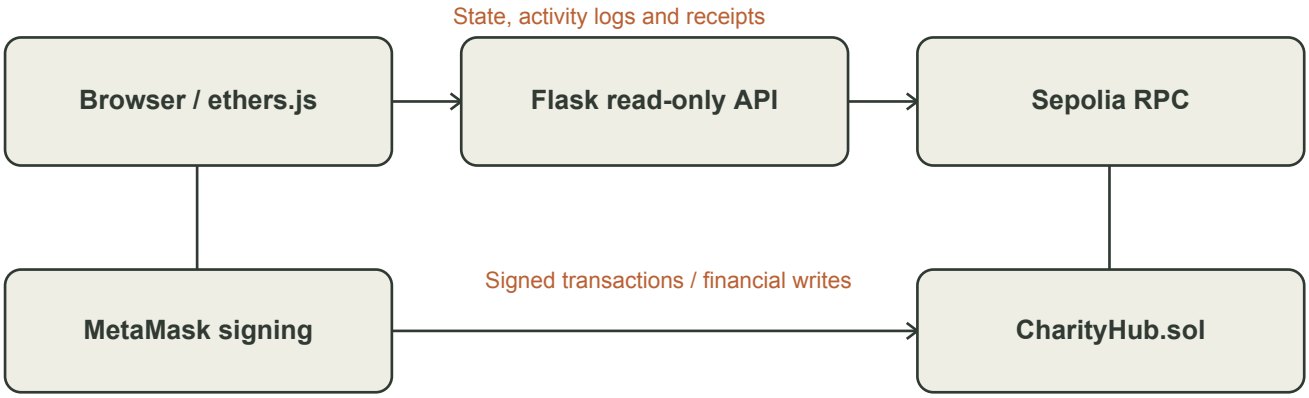
Blockchain does not verify a charity or establish whether withdrawn funds were used as promised. This project therefore labels campaigns as demonstrations, uses test ETH and identifies beneficiaries as wallet addresses. Its real-world problem is financial traceability; its claimed solution is deliberately limited to the on-chain portion of that problem.

3. Objectives and scope

The objectives are to implement payable donation and authorized withdrawal functions; display exact monetary balances and transaction history; support MetaMask and phone-sized screens; integrate Flask without collecting signing keys; validate inputs and explain failures; and evaluate security and usability using reproducible tests. Three seeded community causes make the application immediately understandable after deployment.

The funding model is flexible: donations are final, goals are targets, and beneficiaries can withdraw before a target is reached. There is no automatic refund, investment return or tax receipt. Administrator creation and pausing are explicit trust assumptions. Development and documentation were assisted by ChatGPT/Codex; the student should review the implementation and explain its design choices in the final submission.

4. System architecture and technologies



The browser obtains the ABI, contract address and chain identity from Flask. Public reads use same-origin JSON endpoints, so they do not depend on browser access to third-party RPC services. Financial writes travel from ethers.js to the injected wallet, which prompts the user and submits the signed transaction to the chain. The server never signs transactions or owns user funds.

Layer	Technology and role
Contract	Solidity 0.8.24; optimizer 200 runs; EVM Paris. Standalone source supports Remix without package imports.
Blockchain	Ethereum Sepolia for the intended deployment; Ganache chain 31337 only for disposable development tests.
Frontend	Semantic HTML, responsive CSS, vanilla JavaScript, vendored ethers.js 6.16.0; MetaMask / EIP-1193 wallet interface.
Backend	Flask 3.1.2 and web3.py 7.13.0; Gunicorn with four threads for Render.
Testing	Node built-in test runner, Ganache, Solidity compiler, Python unittest and a synthetic-wallet DOM integration test.
Hosting / source	GitHub source repository; Render Python Web Service. No centralized transaction database.

Read reliability and consistency

The reader verifies the RPC chain ID, nonempty deployed bytecode and contract APP_ID before returning data. A snapshot obtains one latest block and reads all campaign and account values at that block, reducing mismatched totals. Independent calls run in a bounded thread pool. RPC URLs are server configuration, with endpoint failover, request timeouts and a short failure cooldown. A five-second account-scoped cache reduces repetitive reads without mixing donor summaries.

Data boundaries

Monetary values cross the JSON API as wei strings and are displayed using ethers integer conversion. No floating-point number is used to sign a donation. Campaigns, balances and activity are persistent on chain. A browser may remember a pending transaction hash locally, but this is convenience state rather than financial truth. Flask restarts do not erase contract records. Private provider URLs are not exposed in the public configuration API.

5. Smart contract design

ChartityHub separates an immutable administrator from each campaign beneficiary. The constructor creates Community pantry, Learning essentials and Emergency relief, initially assigned to the deployer. New campaigns can use a different beneficiary. There is no proxy or upgrade mechanism; changing the deployed interface requires a new deployment and matching ABI.

Function	Financial behavior / validation
createCampaign	Owner only; title 3-64 UTF-8 bytes, description 10-240 bytes, valid beneficiary, goal over zero and at most 100 ETH, duration 1-365 days, maximum 32 campaigns.
donate	Payable; valid active campaign, positive value up to 1 ETH. Updates campaign totals, platform totals and donor accounting, then records activity.
withdraw	Beneficiary only; pays the entire raised-minus-withdrawn balance. Unrelated wallets, including the owner of another beneficiary campaign, cannot withdraw.
setPaused	Owner only; pauses or resumes new donations. Does not block beneficiary withdrawal.
getHistory	Newest-first global or personal records; limits 1-20 and a continuation cursor. Activity includes actor, amount, campaign, type, time and block.
receive / fallback	Reject accidental direct transfers and unsupported function selectors. Contributions must use donate(campaignId).

Accounting and audit trail

Each campaign stores its target, cumulative raised amount and cumulative withdrawn amount. Available funds equal raised minus withdrawn. Total donations equal the sum of campaign raised amounts; total payouts equal the sum of withdrawn amounts. A donor can contribute multiple times. Goals may be exceeded and another donation is allowed after a payout while the campaign remains active. ActivityRecorded events allow the backend to associate stored records with transaction hashes from their known blocks.

Security properties and trust assumptions

Withdrawals follow checks-effects-interactions: permissions and balance are checked, accounting and activity are updated, then ETH is transferred. A reentrancy guard prevents recursive financial calls. If a beneficiary rejects the transfer, the transaction reverts and all accounting and history effects roll back. Solidity 0.8 arithmetic provides checked overflow behavior. These controls are supported by tests, not an independent audit.

The owner controls listing and pause decisions, and that key has no recovery or transfer mechanism. Beneficiary identity is not verified. A recipient contract that never accepts ETH can leave donations unavailable. Transparent records do not guarantee ethical use after withdrawal. These limitations are part of the application model and should be communicated before donating.

6. Application design and implementation

The interface places campaign descriptions and funding information beside a contribution form. Platform totals distinguish cumulative donations, payouts and funds still available. Each campaign shows the beneficiary and deadline before the donor accepts the final-donation terms. Mobile layouts collapse to a single column, preserve readable forms and keep the history table in its own scrolling area. The phone link opens the Render URL in MetaMask for signing.

Wallet and transaction workflow

Connection is explicit. The controller discovers injected wallets, checks the required chain before signing, obtains the active signer and verifies APP_ID. Changing wallet accounts refreshes personal data and role controls. The server accepts public addresses only for read filtering; actual financial authorization is enforced by the contract using msg.sender, not by the visibility of a button.

State	User feedback and action
Wallet request	Prompt the user to approve or reject in MetaMask; disable concurrent financial actions.
Submitted	Store the transaction hash and show an explorer link. The send response alone is not treated as successful funding.
Confirmed	Receipt status 1 triggers a fresh snapshot and history read. Confirmation is one mined inclusion, not absolute finality.
Reverted	Receipt status 0 explains failure; no donation or payout was recorded, although gas may have been charged.
Read unavailable / delayed	Keep the transaction tracked, prevent immediate duplicate submission, and offer another confirmation check plus explorer/wallet inspection.

Backend APIs and error handling

/api/config supplies the ABI and public network settings. /api/state returns campaign and optional donor summaries. /api/history validates addresses, page size and cursor before reading chain records. /api/receipt validates a transaction hash, checks the receipt destination and distinguishes pending, confirmed and failed transactions. /healthz checks that the web service process is available; it does not establish that RPC reads succeed.

Client validation checks decimal precision and amount bounds, beneficiary addresses, campaign duration and UTF-8 text limits. The contract repeats the important checks because a caller can bypass the page. Contract-controlled strings are rendered with textContent rather than injected HTML. RPC errors return actionable JSON. If a fresh snapshot fails, financial controls are disabled and older visible data is labelled potentially stale. Wallet rejection is explained separately from a mined revert or a missing confirmation read.

7. Testing and local results

Tests use real disposable EVM transactions and a running Flask service. They do not use the student's private keys or claim a public deployment. The contract suite passed 12 scenarios; Node reports 13 passes when including its parent test. The backend passed 10 unit/API scenarios and one real local-RPC integration scenario. A separate local wallet-controller integration exercises the complete UI transaction path with a synthetic EIP-1193 provider.

Evaluation	Observed local result
Successful funding	Clean fixture: 0.007 and 0.003 ETH to pantry, 0.004 ETH to learning; total 0.014 ETH.
Payout and balance	Pantry beneficiary withdrew 0.010 ETH; total available became 0.004 ETH. API and UI matched integer wei accounting.
Authorization / invalid input	Unauthorized admin/payout requests, missing campaigns, zero/over-limit donations and invalid metadata reverted.
Security / failure	Reentrant payout attempt was blocked; rejecting beneficiary rolled back totals and history.
Lifecycle / history	Paused or expired donations rejected; withdrawal remained possible. Newest-first and personal history pagination returned the expected records.
UI and backend	Account-scoped cache, failover, 400/503 JSON errors and local receipt lookup passed. At 390px, document width equalled viewport width.

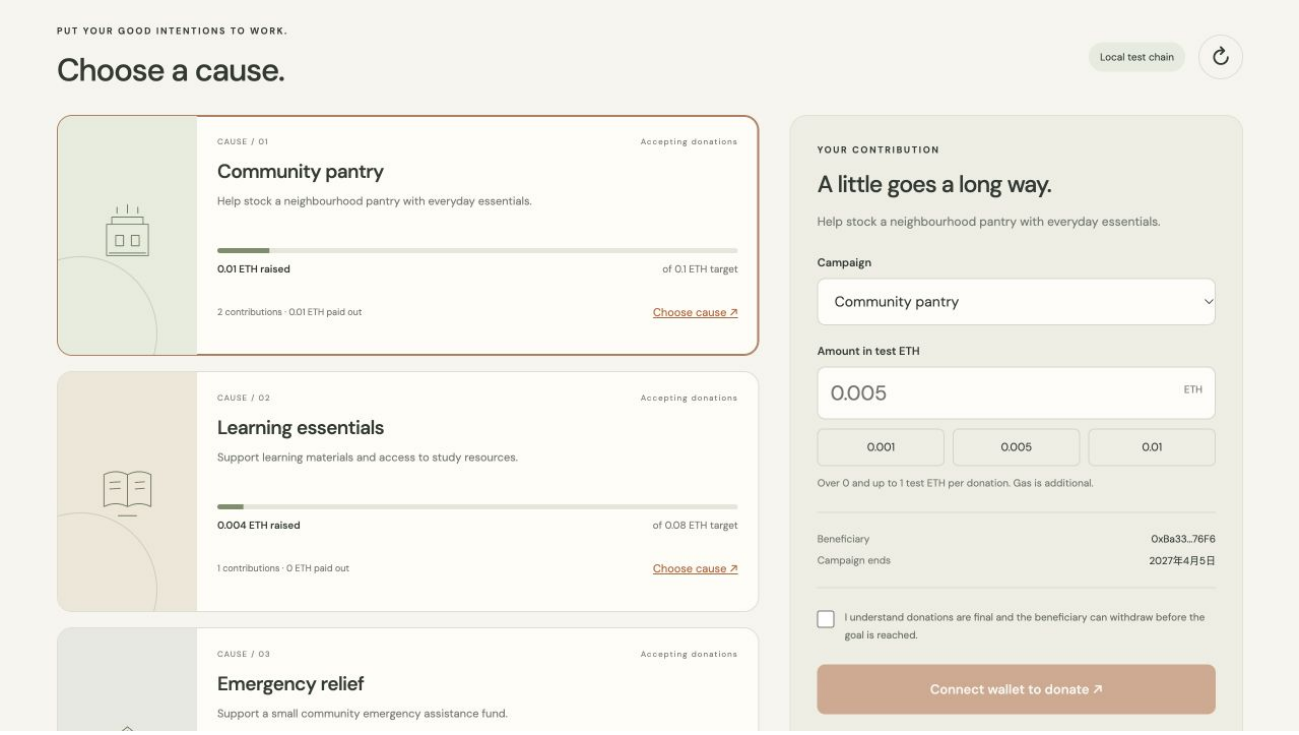


Figure 1. Actual local campaign page connected to Ganache, before the controller test adds further disposable transactions. No public-network result is represented by this screenshot.

The screenshot fixture was intentionally small and reproducible. The separate UI integration mutates a disposable chain to check connection, invalid input, donations, campaign creation, beneficiary account switching, payouts, rejected wallet requests and recovery after a simulated receipt-read outage. A fresh fixture is required before repeating exact baseline-total assertions.

8. Screenshots and usability evidence

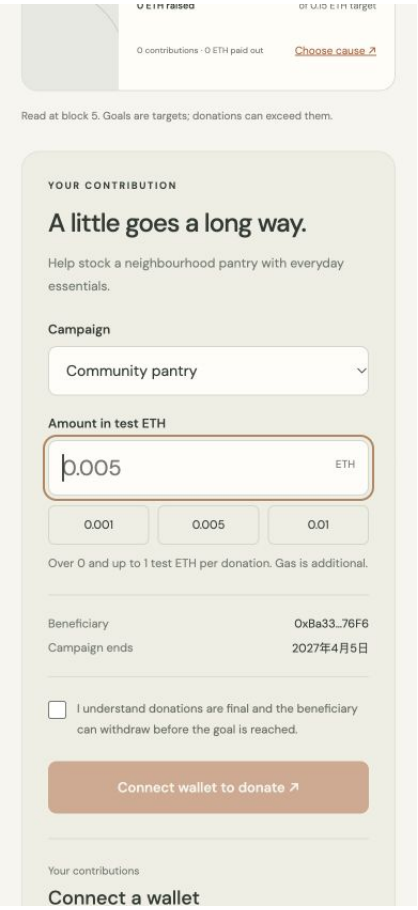


Figure 2. Actual 390 x 844 local mobile viewport. The selected campaign, test-ETH amount, beneficiary, deadline and final-donation consent are accessible in the single-column form.

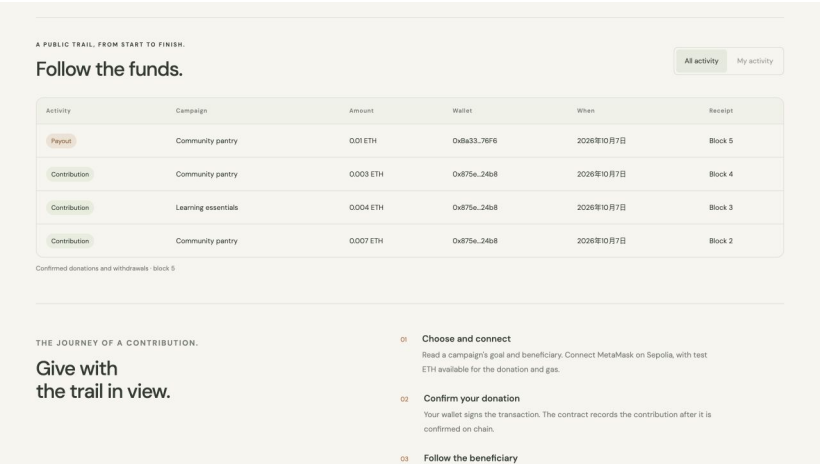


Figure 3. Public local-chain history includes contributions and beneficiary payout. Records carry their chain block and transaction hash; the local UI displays block references because there is no public explorer.

Usability observations

The browser check confirmed that changing the selected cause updates the form and that a preset fills 0.005 ETH. The donation button remains disabled before wallet connection. Phone-size rendering did not produce document-wide horizontal overflow. The interface separates lifetime funds raised from current funds available so a payout is not mistaken for a lost contribution. Wallet and explorer links provide paths to independently inspect a record.

Evidence still required on Sepolia

The student must deploy through Remix and record the contract address and deployment hash, configure Render and record its HTTPS URL, then complete a small successful donation and beneficiary withdrawal with MetaMask. Desktop and MetaMask mobile-browser screenshots should show the successful receipt and updated activity. Also record wallet rejection, invalid input and an unauthorized withdrawal attempt. Local synthetic-wallet tests validate application logic but cannot prove the exact behavior of a specific installed MetaMask version or a real phone.

At this draft stage, the new CharityHub Sepolia deployment and Render deployment have not been performed by this implementation workflow. No public transaction hashes, live performance results or real-asset donations are asserted. Replace or supplement these local screenshots with the final deployed evidence before submission.

9. Evaluation, limitations and conclusion

Challenges encountered

A key reliability issue in DApps is distinguishing transaction submission from chain confirmation when an RPC read fails. The implementation retains pending hashes and allows confirmation retries rather than reporting false success or permitting immediate duplicate donations. Moving public reads to Flask also reduces browser-specific RPC access problems. Local instant-mining tests required disabling ethers provider read caching so pause/resume gas estimates used current state. The final deployed behavior does not bypass wallet validation.

Performance and security evaluation

Campaign listings are bounded to 32 and history pages to 20; receipt-log lookups target known blocks instead of scanning the entire chain. Snapshot reads share a block and use bounded parallel calls and caching. This improves practical read behavior, but no large-scale benchmark was run. Storing full activity records improves simple history retrieval at the cost of transaction gas and permanent storage growth. Public RPC services and free Render instances may add throttling, delays and cold starts.

No seed phrase or private key is stored or requested by Flask. A connected address is not a verified human identity. The contract protects payout permissions and uses a guard plus checks-effects-interactions, but this is a tested academic prototype rather than an audited financial system. Ganache and its transitive dependencies are development-only; Render installs the Python backend and serves the already vendored browser library.

Limitations and future improvements

The platform lacks verified charity registration, fiat conversion, refund rules, multi-signature administration, administrator recovery, recipient-contract compatibility checks and accounting for use after withdrawal. Public descriptions and wallet activity are immutable and can affect privacy. Wallet replacements and cancellations may require manual inspection before stopping pending tracking. Future work could add verified campaign attestations, a multi-signature administrator, explicit recipient acceptance, event indexing for larger histories and user-tested accessibility improvements. Refundable fundraising would require a different financial model, not a cosmetic UI change.

Conclusion

Common Good demonstrates an original donation DApp with financial transfers, contract-enforced permissions, a Flask integration and a mobile-responsive interface. Verified local results show correct monetary accounting, donor and beneficiary flows, and handling of security and input failures. Final assignment completion requires the student to add the actual Remix/Sepolia and Render evidence, review the code and explain the trade-offs in their own presentation of this report.

Primary references and source

Solidity: security considerations

ethers.js v6: providers and contract transactions

MetaMask: Ethereum provider API

Render: deploy a Flask application

Project source repository